

# Experiment Plan: Redundant Belief-State Copies in Transformer Residual Streams

C. L.

March 4, 2026

## 1 Motivation

The ablation experiments (Section 5 of the time-reversal report) established two facts:

1. Ablating the fractal directions causes a *much* larger loss increase than ablating the orthogonal complement, confirming the causal role of the belief-state subspace.
2. Nevertheless, the performance drop from ablating the fractal directions is surprisingly small, especially for models with larger embedding dimensions  $d$ . The model retains substantial predictive ability even after its “primary” belief-state representation has been zeroed out.

This raises a natural question: **why does the model still perform well after the fractal subspace is removed?** Two competing hypotheses:

**H1 (Redundant copies)** The model stores multiple, approximately independent copies of the belief state across different directions in the residual stream. Removing the leading copy (the one found by the first linear probe) still leaves other copies intact. Larger  $d$  allows more copies, explaining the weaker ablation effect.

**H2 (Non-belief information)** The model also exploits features beyond the belief state—e.g., higher-order correlations,  $n$ -gram statistics, or positional patterns—that are not captured by the linear probe to belief states but still contribute to next-token prediction.

H1 is the simpler hypothesis and can be tested directly with existing infrastructure. H2 would require a more involved analysis (e.g., nonlinear probes, or probing for alternative features). We address H1 first.

## 2 Proposed Experiment

### 2.1 Core Idea

If redundant copies exist, then projecting out the fractal subspace found by the first probe should *not* destroy all belief-state information in the residual stream. A second linear probe, fitted on the *remnant* activations (the component orthogonal to the first probe’s row space), should still achieve a non-trivial  $R^2$ .

Concretely, let  $W^{(1)} \in \mathbb{R}^{N \times d}$  be the weight matrix of the first linear probe (the regressor’s `coef_`). Its row space defines the fractal subspace  $\mathcal{F}_1 \subseteq \mathbb{R}^d$ . The orthogonal projector onto  $\mathcal{F}_1^\perp$  is

$$P_\perp = I - V_r V_r^\top, \tag{1}$$

where  $V_r \in \mathbb{R}^{d \times r}$  contains the first  $r = \text{rank}(W^{(1)})$  right singular vectors of  $W^{(1)}$ . The remnant activation is  $\tilde{h}_t = P_{\perp} h_t$ .

The experiment then fits a second linear probe:  $W^{(2)}\tilde{h}_t + b^{(2)} \approx \alpha_t$ .

## 2.2 Iterative Procedure

This can be iterated: project out the combined subspace of probes 1 and 2, fit probe 3, and so on. At each step  $k$ :

1. Let  $\mathcal{F}_{1:k-1}$  be the union of all previously found fractal subspaces.
2. Compute  $\tilde{h}_t^{(k)} = P_{\mathcal{F}_{1:k-1}^{\perp}} h_t$ .
3. Fit a linear probe on  $\tilde{h}_t^{(k)}$  and record  $R_k^2$ .
4. Stop when  $R_k^2$  drops below a threshold (e.g.,  $R_k^2 < 0.1$ ) or the remnant subspace is exhausted ( $d - \sum_{j < k} r_j \leq 0$ ).

The sequence  $R_1^2, R_2^2, \dots$  directly answers H1: if it decays slowly, there are many copies; if  $R_2^2 \approx 0$ , there is essentially one copy.

## 2.3 Experimental Matrix

The experiment should be run across models with varying  $d$  to test the prediction that larger  $d$  supports more redundant copies:

| Variable       | Values                                                |
|----------------|-------------------------------------------------------|
| HMM process    | Mess3 (3 states, 3 tokens)                            |
| Model pool     | Existing trained models from the architecture sweep   |
| Embedding dims | All available $d \in \{4, 8, 16, 32, \dots\}$         |
| Layers probed  | All layers (layer-wise analysis)                      |
| Position       | Last position only ( <code>use_last_pos=True</code> ) |
| Max iterations | Until $R^2 < 0.1$ or subspace exhausted               |

Table 1: Experimental parameters.

## 2.4 Deliverables

1.  **$R^2$  decay curves:**  $R_k^2$  vs. iteration  $k$ , one curve per model (or per  $d$ ), at the best-performing layer.
2. **Total explained dimensions:** The total number of directions  $\sum_k r_k$  needed before belief-state information is exhausted, as a function of  $d$ . If H1 holds, this should scale with  $d$ .
3. **Simplex visualisations:** Plot the predicted belief states from probe  $k$  on the simplex, to see whether later copies recover the same fractal geometry or a degraded version.
4. **Comparison with ablation:** After removing all  $K$  identified fractal subspaces, repeat the ablation experiment. If H1 is the full story, the model should now show a much larger performance drop.

### 3 Relation to H2

If the iterative probing exhausts quickly ( $R_2^2 \approx 0$ ) but the ablation deficit remains small, this would be evidence *against* H1 and *for* H2. In that case, the follow-up would be to characterise what non-belief features the model uses. Possible directions:

- Nonlinear probes (MLP) on the remnant activations to test whether belief-state information is stored nonlinearly.
- Probing for  $n$ -gram statistics or positional features in the orthogonal complement.
- Logit lens analysis: project the remnant activations through the unembedding matrix to see what token predictions they support.

These are out of scope for the current experiment but would be natural next steps.

### 4 Implementation Notes

The experiment requires no changes to the existing probing code (`old_src/pca.py`). The pipeline is:

1. Extract residual-stream activations using `extract_residual_stream` (already implemented).
2. Fit the first probe using `learn_affine_mapping` (already implemented).
3. Compute the remnant:  $\tilde{h} = h - V_r V_r^\top h$  using the SVD of `regressor.coef_` (the SVD is already computed in `get_directions` in `old_src/ablation.py`).
4. Call `learn_affine_mapping` again on the remnant.
5. Iterate.

The only new code is a short loop that alternates projection and probing. This can be implemented in a notebook or a lightweight script.